

Controlling Software Environment

On the Importance of Software Environments

Package Management

Fundamentals of Operating System Architecture

Containers

Docker Tutorial

Argh... damned computers

- Alice: *"I got 3.123123"* Bob: *"I got segfault"*

Argh... damned computers

- Alice: *"I got 3.123123"* Bob: *"I got segfault"*
- *"Damned! It used to work!!! Whenever I upgrade my computer, things break so I try to stay away from this"* 😞
- *"Anyway, I don't have the root password"* *"The what?..."*

Argh... damned computers

- Alice: *"I got 3.123123"* Bob: *"I got segfault"*
- *"Damned! It used to work!!! Whenever I upgrade my computer, things break so I try to stay away from this"* 😞
- *"Anyway, I don't have the root password"* *"The what?..."*
- *"Whenever trying the code of my colleague, I had to install Foo but I broke everything and now neither his code nor mine works!"* 😞

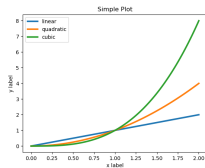
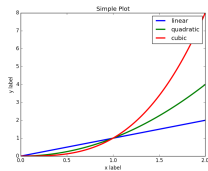
Argh... damned computers

- Alice: *"I got 3.123123"* Bob: *"I got segfault"*
- *"Damned! It used to work!!! Whenever I upgrade my computer, things break so I try to stay away from this"* 😞
- *"Anyway, I don't have the root password"* *"The what?..."*
- *"Whenever trying the code of my colleague, I had to install Foo but I broke everything and now neither his code nor mine works!"* 😞
- *"But hey! Here is my code, feel free to play with it! I'm doing open science"* 😊

Seriously ? How come all this is so painful ?

Backwards compatibility

- Software environment evolution



Backwards compatibility

- Software environment evolution
- Software evolution and OS heterogeneity

The Effects of FreeSurfer Version, Workstation Type, and Macintosh Operating System Version on Anatomical Volume and Cortical Thickness Measurements (PLOS ONE, 2012)

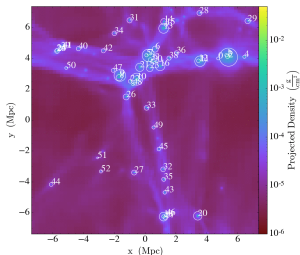
*Significant differences in volume and cortical thickness were revealed **across FreeSurfer versions**. In addition, less pronounced differences were found **between the Mac and HP workstations** and **between Mac OSX 10.5 and OSX 10.6**.*

Backwards compatibility

- Software environment evolution
- Software evolution and OS heterogeneity
- Impact of the compiler

Assessing Reproducibility: An Astrophysical Example of Computational Uncertainty in the HPC Context (ResCuE-HPC, 2018)

Compiler	Optim.	Largest Halo Avg Mass.	Std. Err	Walltime
gcc@6.2.0	None	2.273E46	1.069E44	22h
gcc@6.2.0	Normal	2.266E46	1.218E44	10h
gcc@6.2.0	High	2.275E46	1.199E44	9h
intel@16.0.3	None	2.271E45	1.587E44	39h
intel@16.0.3	Normal	4.330(45)	1.248E44	7h
intel@16.0.3	High	2.268E46	1.414E44	6h
cce@8.5.5	Low	4.311(45)	1.353E44	16h
cce@8.5.5	Normal	2.271E46	1.261E44	6h
cce@8.5.5	High	2.272E46	1.341E44	5h

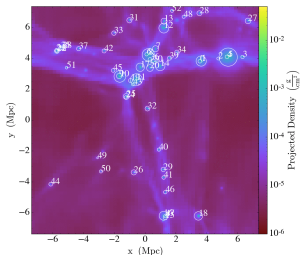


Backwards compatibility

- Software environment evolution
- Software evolution and OS heterogeneity
- Impact of the compiler

Assessing Reproducibility: An Astrophysical
Example of Computational Uncertainty in the HPC
Context (ResCuE-HPC, 2018)

Compiler	Optim.	Largest Halo Avg Mass.	Std. Err	Walltime
gcc@6.2.0	None	2.273E46	1.069E44	22h
gcc@6.2.0	Normal	2.266E46	1.218E44	10h
gcc@6.2.0	High	2.275E46	1.199E44	9h
intel@16.0.3	None	2.271E45	1.587E44	39h
intel@16.0.3	Normal	4.330(45)	1.248E44	7h
intel@16.0.3	High	2.268E46	1.414E44	6h
cce@8.5.5	Low	4.311(45)	1.353E44	16h
cce@8.5.5	Normal	2.271E46	1.261E44	6h
cce@8.5.5	High	2.272E46	1.341E44	5h

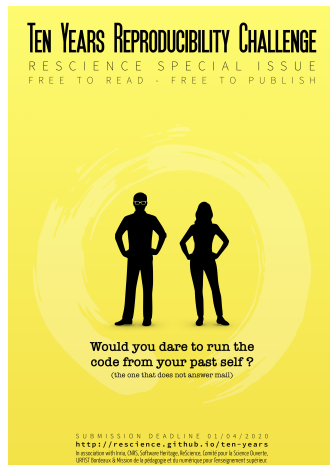


Backwards compatibility

- Software environment evolution
- Software evolution and OS heterogeneity
- Impact of the compiler

Assessing Reproducibility: An Astrophysical
Example of Computational Uncertainty in the HPC
Context (ResCuE-HPC, 2018)

Compiler	Optim.	Largest Halo Avg Mass.	Std. Err	Walltime
gcc@6.2.0	None	2.273E46	1.069E44	22h
gcc@6.2.0	Normal	2.266E46	1.218E44	10h
gcc@6.2.0	High	2.275E46	1.199E44	9h
intel@16.0.3	None	2.271E45	1.587E44	39h
intel@16.0.3	Normal	4.330(45)	1.248E44	7h
intel@16.0.3	High	2.268E46	1.414E44	6h
cce@8.5.5	Low	4.311(45)	1.353E44	16h
cce@8.5.5	Normal	2.271E46	1.261E44	6h
cce@8.5.5	High	2.272E46	1.341E44	5h



<http://rescience.github.io/ten-years/>

Complex ecosystems

```
1 import matplotlib
2 print(matplotlib.__version__)
```

3.1.2

Complex ecosystems

```
1 import matplotlib
2 print(matplotlib.__version__)
```

3.1.2

```
1 apt show python3-matplotlib
```

Package: python3-matplotlib

Version: 3.1.2-2

Priority: optional

Section: python

Source: matplotlib

Maintainer: Sandro Tosi <morph@debian.org>

Installed-Size: 15.3 MB

Depends: python3-dateutil, python-matplotlib-data (>= 3.1.2-2), python3-pyparsing (>= 1.4), libjs-jquery, libjs-jquery-ui, python3-numpy (>= 1:1.16.0-rc1), python3-numpy-abi9, python3 (< 3.9), python3 (>= 3.7~), python3-cycler (>= 0.10.0), python3-kiwisolver, python3:any, libc6 (>= 2.29), libfreetype6 (>= 2.2.1), libgcc-s1 (>= 3.0), libpng16-16 (>= 1.6.2-1), libstdc++6 (>= 5.2)

Recommends: python3-pil, python3-tk

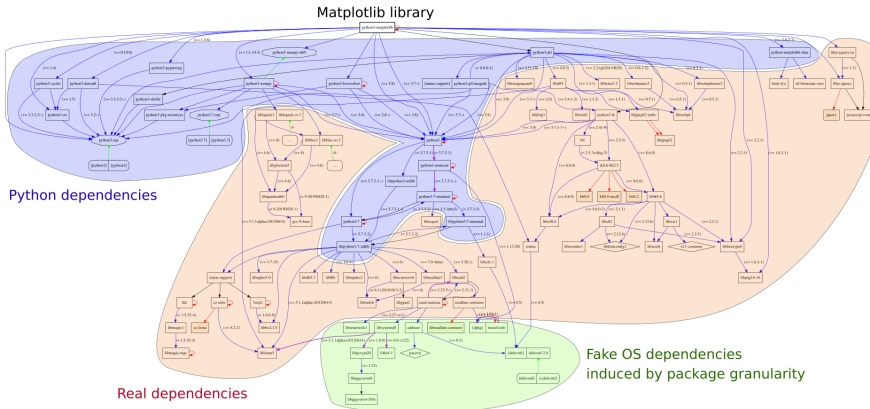
Suggests: dvipng, ffmpeg, gir1.2-gtk-3.0, ghostscript, inkscape, ipython3, libcommon, python-matplotlib-doc, python3-cairocffi, python3-gi, python3-gi-cairo, python3-gobject, python3-nose, python3-pyqt5, python3-scipy, python3-sip, python3-tornado, texlive-extra-utils, texlive-latex-extra, ttf-staypuft

Complex ecosystems

```
1 import matplotlib
2 print(matplotlib.__version__)
```

3.1.2

```
1 apt show python3-matplotlib
```



Non standard ecosystems

No standard

- Linux (`apt`, `rpm`, `yum`), MacOS X (`brew`, `MacPorts`, `Fink`), Windows (?)
- Neither for installation nor for retrieving the information... 😞

```
1 import sys
2 print(sys.version)
3 import matplotlib
4 print(matplotlib.__version__)
5 import pandas as pd
6 print(pd.__version__)
```

```
3.7.6 (default, Jan 19 2020, 22:34:52)
[GCC 9.2.1 20200117]
3.1.2
0.25.3
```

```
1 library(ggplot2)
2 sessionInfo()
```

```
R version 3.6.3 RC (2020-02-21 r77847)
Platform: x86_64-pc-linux-gnu (64-bit)
Running under: Debian GNU/Linux bullseye/sid
```

```
Matrix products: default
```

```
BLAS: /usr/lib/x86_64-linux-gnu/atlas/libblas.so.3.10.3
LAPACK: /usr/lib/x86_64-linux-gnu/atlas/liblapack.so.3.10.3
```

```
locale:
[1] C
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods
```

```
other attached packages:
```

```
[1] ggplot2_3.2.1
```

```
loaded via a namespace (and not attached):
```

```
[1] Rcpp_1.0.3      withr_2.1.2      crayon_1.3.4      dplyr_1.0.2
[5] assertthat_0.2.1 grid_3.6.3       R6_2.4.1          lifecycle_0.2.0
[9] gtable_0.3.0    magrittr_1.5     scales_1.1.0      pillar_1.4.3
[13] rlang_0.4.4     lazyeval_0.2.2   glue_1.3.1        purrr_0.3.3
[17] munsell_0.5.0   compiler_3.6.3   pkgconfig_2.0.3   colorspace_1.4-1
[21] tidyr_1.0.0     tibble_2.1.3     tidyselect_1.0.0  readr_1.3.1
```

Argh... damned computers

- *"Whenever I upgrade my computer, things break so I try to stay away from this"* 😞
- *"Whenever trying the code of my colleague, I had to install Foo but I broke everything and now neither his code nor mine works!"* 😞
- *"But hey! Here is my code, feel free to play with it! I'm doing open science"* 😊

Argh... damned computers

- *"Whenever I upgrade my computer, things break so I try to stay away from this"* 😞
- *"Whenever trying the code of my colleague, I had to install Foo but I broke everything and now neither his code nor mine works!"* 😞
- *"But hey! Here is my code, feel free to play with it! I'm doing open science"* 😊

Are you really aware of your dependencies ?

- No one will ever run/use your code if it isn't easy to install
- No one will ever manage to run your code if you don't document how to run it
- Others (even you) are unlikely to get the same results unless you automate the execution

Solutions we will cover

1. Work in an isolated environment
2. Automate the construction of this environment
3. Use reliable package sources
4. Share this environment
5. Check whether your code runs on an other machine

Controlling Software Environment

On the Importance of Software Environments

Package Management

Fundamentals of Operating System Architecture

Containers

Docker Tutorial

Installing software from sources

- `configure` (or `cmake` or whatever)
 - detect and check dependencies
 - test features (not versions)!
 - generates header files and makefiles

Installing software from sources

- `configure` (or `cmake` or whatever)
 - detect and check dependencies
 - test features (not versions)!
 - generates header files and makefiles
- `make`
 - compile and link
 - create shared libraries and executables

Installing software from sources

- `configure` (or `cmake` or whatever)
 - detect and check dependencies
 - test features (not versions)!
 - generates header files and makefiles
- `make`
 - compile and link
 - create shared libraries and executables
- `make install` (as root)
 - copy shared libraries, headers and executables

Installing software from sources

- `configure` (or `cmake` or whatever)
 - detect and check dependencies
 - test features (not versions)!
 - generates header files and makefiles
- `make`
 - compile and link
 - create shared libraries and executables
- `make install` (as root)
 - copy shared libraries, headers and executables

This is often:

1. tedious, time consuming, error prone, frustrating, ...
2. incoherent
 - Filename conflicts during `install`?
 - Will `my_super_code` still work after upgrading `libexotic.so`?

Installing binary packages (as root)

```
1 dpkg -i rstudio_last_version.deb
```

- No need to recompile
- `dpkg` will ensure version coherency, warn about conflicts

Depends: libedit2, libssl1.0.0 | libssl1.0.2 | libssl1.1, libclang-dev, libc6 (>= 2.7)

Installing binary packages (as root)

```
1 dpkg -i rstudio_last_version.deb
```

- No need to recompile
- `dpkg` will ensure version coherency, warn about conflicts
 - Depends: libedit2, libssl1.0.0 | libssl1.0.2 | libssl1.1, libclang-dev, libc6 (>= 2.7)

```
1 apt install r-cran-tidyverse
```

- Will download from trusted sources all the `.deb` packages you need

Installing binary packages (as root)

```
1 dpkg -i rstudio_last_version.deb
```

- No need to recompile
- `dpkg` will ensure version coherency, warn about conflicts
 - Depends: libedit2, libssl1.0.0 | libssl1.0.2 | libssl1.1, libclang-dev, libc6 (>= 2.7)

```
1 apt install r-cran-tidyverse
```

- Will download from trusted sources all the `.deb` packages you need

The **Debian** package manager:

- distinguishes between **build** and **binary** dependencies
- applies patches to make correct compiling/installation
- handles dependencies, conflicts, alternatives, ...
- handles potential security and quality issues

Debian installation essentials

```
1 apt update # Update the package list
2 apt search neuroscience # or apt-cache
3 apt show <package_name>
4 apt install <package_name>
5 apt upgrade # Upgrade all your packages
6 apt clean # Delete all the .deb you've downloaded
7 apt remove --purge <package_name>
```

Any drawback with binary installation ?

- ~~Not specifically optimized for your machine (Gentoo)~~

Any drawback with binary installation ?

- ~~Not specifically optimized for your machine (Gentoo)~~
- Binary package and compiling may incur "fake" dependencies
 - `Depends: libc6 (>= 2.7)` How do you know the future?
 - Some requirements may be too strong

Any drawback with binary installation ?

- ~~Not specifically optimized for your machine (Gentoo)~~
- Binary package and compiling may incur "fake" dependencies
 - `Depends: libc6 (>= 2.7)` How do you know the future?
 - Some requirements may be too strong
- Code comes in "bulk" (latex compiler, documentation, etc.)

Any drawback with binary installation ?

- ~~Not specifically optimized for your machine (Gentoo)~~
- Binary package and compiling may incur "fake" dependencies
 - `Depends: libc6 (>= 2.7)` How do you know the future?
 - Some requirements may be too strong
- Code comes in "bulk" (latex compiler, documentation, etc.)
- `apt` or other similar tools
 - easy installation of "some package" at a given date
 - installing a specific version or combination is hard

Any drawback with binary installation ?

- ~~Not specifically optimized for your machine (Gentoo)~~
- Binary package and compiling may incur "fake" dependencies
 - `Depends: libc6 (>= 2.7)` How do you know the future?
 - Some requirements may be too strong
- Code comes in "bulk" (latex compiler, documentation, etc.)
- `apt` or other similar tools
 - easy installation of "some package" at a given date
 - installing a specific version or combination is hard
- *What if my distribution is not up-to-date ?*
 - Ubuntu LTS or Debian stable 😊 vs. testing/unstable 😞

Any drawback with binary installation ?

- ~~Not specifically optimized for your machine (Gentoo)~~
- Binary package and compiling may incur "fake" dependencies
 - `Depends: libc6 (>= 2.7)` How do you know the future?
 - Some requirements may be too strong
- Code comes in "bulk" (latex compiler, documentation, etc.)
- `apt` or other similar tools
 - easy installation of "some package" at a given date
 - installing a specific version or combination is hard
- *What if my distribution is not up-to-date ?*
 - Ubuntu LTS or Debian stable 😊 vs. testing/unstable 😐
- *What if a given software is not packaged ?*
 - Use python/R/specific installation mechanism
 - Use additional alternative distribution (e.g., conda)

Be prepared to mess everything up! 😞

Using ad hoc installation procedures

R `install.packages("ggplot2")`

- Linux/Mac: builds from sources
 - hence requires a compiler and all the right libraries...
- Windows: downloads binary image

Using ad hoc installation procedures

R `install.packages("ggplot2")`

- Linux/Mac: builds from sources
 - hence requires a compiler and all the right libraries...
- Windows: downloads binary image
- This version requires a more recent R 😞
 1. Upgrade R.
 2. (Try to) reinstall through R all the packages you had previously installed! 😞

Using ad hoc installation procedures

R `install.packages("ggplot2")`

- Linux/Mac: builds from sources
 - hence requires a compiler and all the right libraries...
- Windows: downloads binary image
- `This version requires a more recent R` 😞
 1. Upgrade R.
 2. (Try to) reinstall through R all the packages you had previously installed! 😞

Python

- `pip install numpy` does not check for conflict with previously installed packages!!!

Using ad hoc installation procedures

R `install.packages("ggplot2")`

- Linux/Mac: builds from sources
 - hence requires a compiler and all the right libraries...
- Windows: downloads binary image
- This version requires a more recent R 😞
 1. Upgrade R.
 2. (Try to) reinstall through R all the packages you had previously installed! 😞

Python

- `pip install numpy` does not check for conflict with previously installed packages!!!
- [Ana|Mini]conda: python and R distribution for scientific computing on Windows, MacOS X, Linux
 - `conda list --export > requirements.txt`
 - `conda create --name <envname> --file requirements.txt` or `conda install pandas=0.13.1`
 - Recreating an old installation is not so simple 😞

Using ad hoc installation procedures

R `install.packages("ggplot2")`

- Linux/Mac: builds from sources
 - hence requires a compiler and all the right libraries...
- Windows: downloads binary image
- This version requires a more recent R 😞
 1. Upgrade R.
 2. (Try to) reinstall through R all the packages you had previously installed! 😞

Python

- `pip install numpy` does not check for conflict with previously installed packages!!!
- [Ana|Mini]conda: python and R distribution for scientific computing on Windows, MacOS X, Linux
 - `conda list --export > requirements.txt`
 - `conda create --name <envname> --file requirements.txt` or `conda install pandas=0.13.1`
 - Recreating an old installation is not so simple 😞

Mixing distributions is generally a terrible idea!

Why are you focusing on Debian ?

Three good reasons

1. It is an open source leader (GNU/Linux)
 - Reproducibility on Mac OS X or Windows is almost impossible
 - **main** is 100% free, **non-free** packages are optional
 - Initial release: 1993!

Why are you focusing on Debian ?

Three good reasons

1. It is an open source leader (GNU/Linux)
 - Reproducibility on Mac OS X or Windows is almost impossible
 - **main** is 100% free, **non-free** packages are optional
 - Initial release: 1993!
2. **Debian reproducible builds**
 - Most packages built in sid today are reproducible under a fixed build-path and environment.
 - CI builds and immediately rebuilds packages to detect problems related to timestamps, file ordering, CPU usage, (pseudo-)randomness, etc.

Why are you focusing on Debian ?

Three good reasons

1. It is an open source leader (GNU/Linux)
 - Reproducibility on Mac OS X or Windows is almost impossible
 - **main** is 100% free, **non-free** packages are optional
 - Initial release: 1993!
2. **Debian reproducible builds**
 - Most packages built in sid today are reproducible under a fixed build-path and environment.
 - CI builds and immediately rebuilds packages to detect problems related to timestamps, file ordering, CPU usage, (pseudo-)randomness, etc.
3. The **snapshot archive** is a wayback machine

```
deb      https://snapshot.debian.org/archive/debian/20091004T111800Z/ lenny main
deb-src  https://snapshot.debian.org/archive/debian/20091004T111800Z/ lenny main
```

Why are you focusing on Debian ?

Three good reasons

1. It is an open source leader (GNU/Linux)
 - Reproducibility on Mac OS X or Windows is almost impossible
 - **main** is 100% free, **non-free** packages are optional
 - Initial release: 1993!
2. **Debian reproducible builds**
 - Most packages built in sid today are reproducible under a fixed build-path and environment.
 - CI builds and immediately rebuilds packages to detect problems related to timestamps, file ordering, CPU usage, (pseudo-)randomness, etc.
3. The **snapshot archive** is a wayback machine

```
deb      https://snapshot.debian.org/archive/debian/20091004T111800Z/ lenny main
deb-src  https://snapshot.debian.org/archive/debian/20091004T111800Z/ lenny main
```

Drawback

- Not as ~~up-to-date~~ diverse as Anaconda
- Mixing versions is less flexible

Controlling Software Environment

On the Importance of Software Environments

Package Management

Fundamentals of Operating System Architecture

Containers

Docker Tutorial

Foreword

Open Science $\Rightarrow$ Open Source $\Rightarrow$ Linux

Using other OS (Mac, Windows) **is** possible but will generally be more tedious

Beyond *Ideology*, let's explain why from a technical point of view

What does a program need ?



Program = code + ...

What does a program need ?



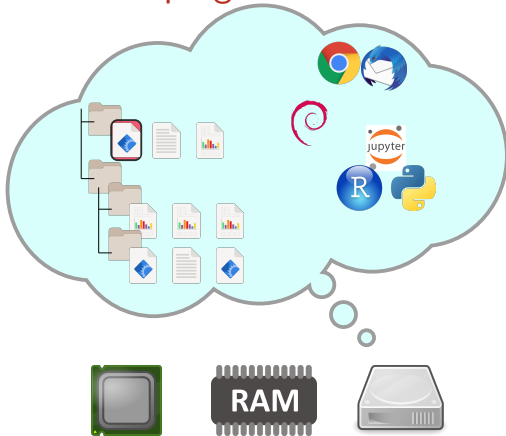
Program = code + ...

What does a program need ?



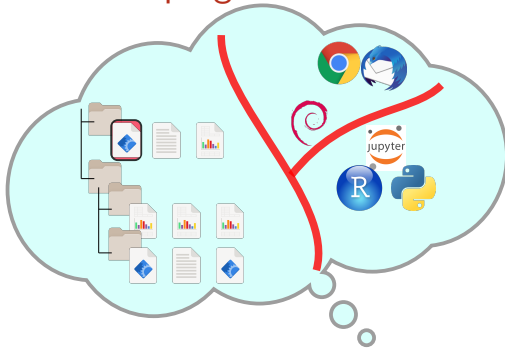
Program = code + ...

What does a program need ?



Program = code + ...

What does a program need ?

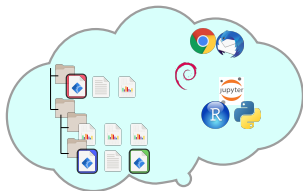
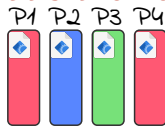


Program = code + ...

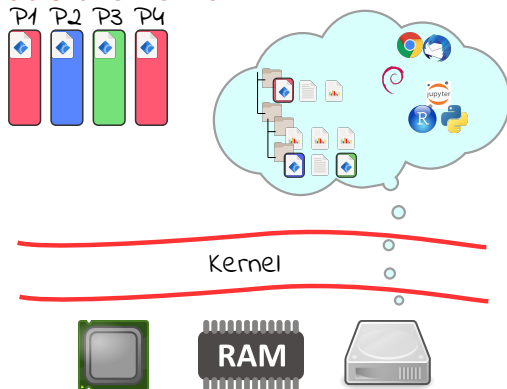
What's the kernel ?



What's the kernel ?



What's the kernel ?



The kernel

Mediates the access to physical resources through **system calls**

- I want to **read/write** some data in a file
- I need more memory (**brk**), virtual address translation
- I run until I'm preempted

Everything is shared, including the software environment

Controlling Software Environment

On the Importance of Software Environments

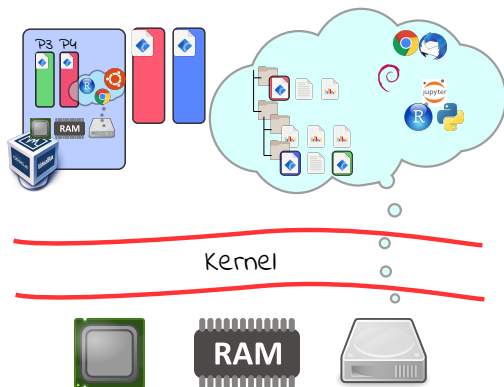
Package Management

Fundamentals of Operating System Architecture

Containers

Docker Tutorial

Virtual Machines



- A virtual machine emulates a whole system
 - Slow
 - Large $\leadsto$ difficult to share
- Communication with the guest system ? Through the "network"

Chroot, Namespaces, and Control groups



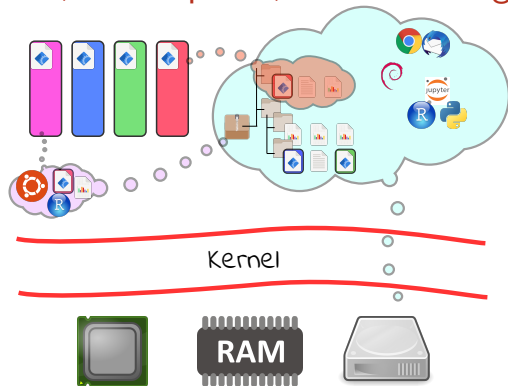
- **chroot**: tweak the root filesystem

Chroot, Namespaces, and Control groups



- **chroot**: tweak the root filesystem

Chroot, Namespaces, and Control groups



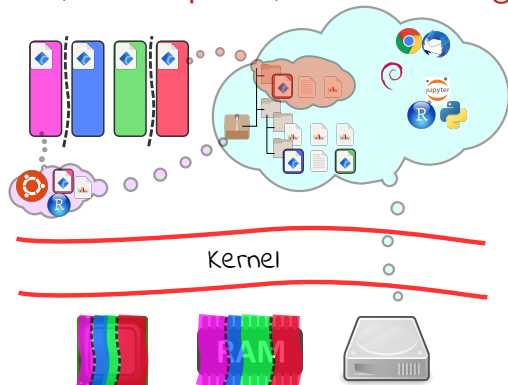
- **chroot**: tweak the root filesystem

Chroot, Namespaces, and Control groups



- **chroot**: tweak the root filesystem
- **namespaces**: isolate process from each others

Chroot, Namespaces, and Control groups



- **chroot**: tweak the root filesystem
- **namespaces**: isolate process from each others
- **cgroups**: limits, accounts for, and isolates resource usage

Containers

Containers = `chroot` + `namespaces` + `cgroups` + ... made easy

- Images only comprise the useful code (~~kernel, drivers~~)
- Process keep using syscalls as usual $\rightsquigarrow$ very lightweight

Containers

Containers = `chroot` + `namespaces` + `cgroups` + ... made easy

- Images only comprise the useful code (~~kernel, drivers~~)
- Process keep using syscalls as usual $\rightsquigarrow$ very lightweight
- Communication with the guest system: through the file system
 - Software environment in the image (as small as possible)
 - Data (input and output) in the guest OS

Containers

Containers = `chroot` + `namespaces` + `cgroups` + ... made easy

- Images only comprise the useful code (~~kernel, drivers~~)
- Process keep using syscalls as usual $\rightsquigarrow$ very lightweight
- Communication with the guest system: through the file system
 - Software environment in the image (as small as possible)
 - Data (input and output) in the guest OS

Docker containers  :

- Running as easy as `docker run <img> <cmd>`
- Building images: `docker build -f <Dockerfile>`
- Sharing through the Docker Hub: `docker pull/push <img>`

Alternatives to containers

Python `virtualenv`, `pipenv`, `conda environments`, ...

- install python packages in specific directories
- `freeze` is possible but:
 - permeable (what about other scripts, codes, ... ?)
 - python-only (what about `libblas`?)

Alternatives to containers

Python `virtualenv`, `pipenv`, `conda environments`, ...

- install python packages in specific directories
- `freeze` is possible but:
 - permeable (what about other scripts, codes, ... ?)
 - python-only (what about `libblas`?)

GUIX/NiX Full-fledged *functional* package manager

- Native support for environment (*à la git*)
- Isolation through `--pure`
- Recompile from source (cache recommended)
 - $\rightsquigarrow$ more flexibility for combination and customization
- Limited set of packages compared to Debian (`not really`) or Anaconda

Limitations and potential issues

- Linux through docker over Mac or Windows
 - Incompatible syscalls $\rightsquigarrow$ Docker in a VM $\Rightarrow$ Sloooooow I/O
 - Windows10 (Windows Subsystem for Linux): compatible syscalls...

Limitations and potential issues

- Linux through docker over Mac or Windows
 - Incompatible syscalls $\rightsquigarrow$ Docker in a VM $\Rightarrow$ Sloooooow I/O
 - Windows10 (Windows Subsystem for Linux): compatible syscalls...
- Old linux over recent linux
 - What if the syscall changed ?!?
 - Very stable but may require `vsyscall=emulate`
- Bootstrapping: use old `.deb` files with a recent `dpkg`
- Networking: old gpg keys, format compatibility, https, ...

Limitations and potential issues

- Linux through docker over Mac or Windows
 - Incompatible syscalls $\rightsquigarrow$ Docker in a VM $\Rightarrow$ Sloooooow I/O
 - Windows10 (Windows Subsystem for Linux): compatible syscalls...
- Old linux over recent linux
 - What if the syscall changed ?!?
 - Very stable but may require `vsyscall=emulate`
- Bootstrapping: use old `.deb` files with a recent `dpkg`
- Networking: old gpg keys, format compatibility, https, ...

Not perfect but (if Linux-only) Docker is:

- Easy to use and set up
- Lightweight
- Robust

Controlling Software Environment

On the Importance of Software Environments

Package Management

Fundamentals of Operating System Architecture

Containers

Docker Tutorial

Goals of the tutorial

`https://gitlab.inria.fr/learninglab/mooc-rr/mooc-rr-ressources/blob/master/module5/ressources/docker\_tutorial\_fr.org`

1. Getting familiar with Docker
 - First steps
 - Configuration for a daily usage
2. Building, automating and sharing your own environment
3. Using Docker in a Continuous Integration context
4. Try doing the same with GUIX (for the brave 😊)

Docker Tips

- Build / copy (checkpoint in the Dockerfile)
- Freeze sources with `debian.snapshots`
- Clean build-dependencies before sharing your image
- Avoid running `apt-get` in CI images